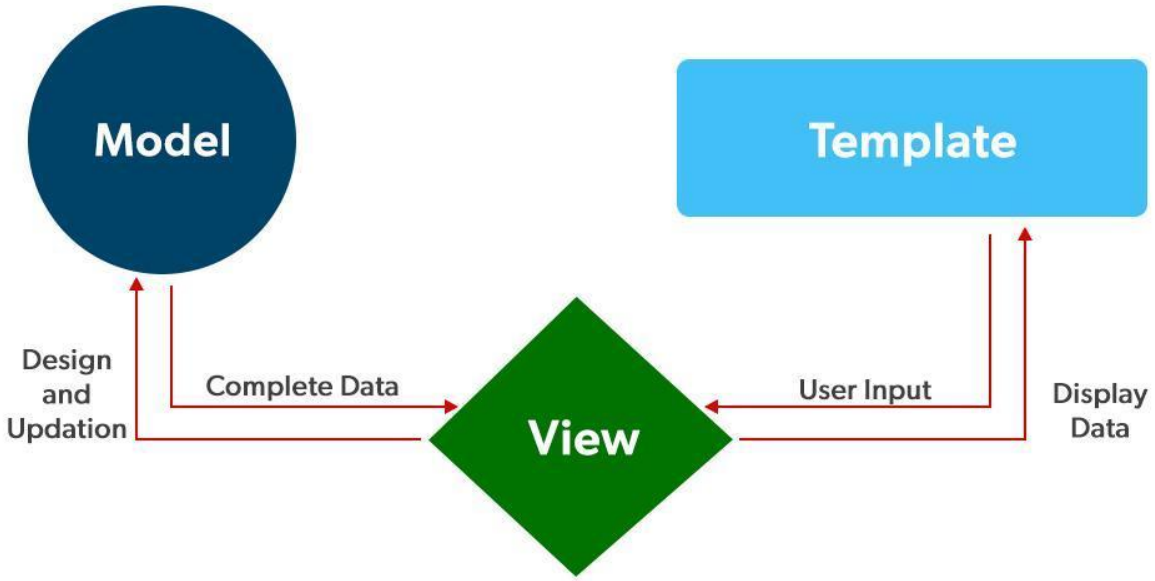
Unit-5
Django Framework

MVT Architecture
Django follows the **MVT (Model-View-Template)** architectural pattern, which is a variation of the traditional MVC (Model-View-Controller) design pattern used in web development.
This pattern separates the application into three main components:
1. Model
Model acts as the data layer of an application. It defines the structure of the database and handles data-related logic.
- Models typically represent database tables.
- Responsible for querying, inserting, updating, and deleting data.
- Usually backed by relational databases such as MySQL, PostgreSQL, or SQLite.

2. View
View contains the business logic of the application. It processes incoming user requests and prepares the data that needs to be displayed.
- Interacts with models to retrieve or update data.
- Passes the prepared data to templates for rendering.
- Implemented as Python functions or class-based views that return HTTP responses.

3. Template
Template is responsible for presenting data to the user. It combines static content with dynamic data using template syntax.
- Contains HTML, CSS, and JavaScript for the user interface.
- Uses Django Template Language (DTL) to insert dynamic data.
- Displays data passed from views in a structured format.



Introduction to Django framework

- Django is a back-end server side web framework.
- Django is free, open source and written in Python.
- Django was design and developed by **Lawrence journal world in 2003** and publicly released under BSD license in July 2005.
- Django makes it easier to build web pages using Python.
- Django emphasizes reusability of components, also referred to **as DRY (Don't Repeat Yourself)**, and comes with ready-to-use features like login system, database connection and CRUD operations (Create Read Update Delete).

Django Project Installation in Virtual Environment

To create virtual environment and install Django, use following commands one by one:

- python -m venv env_name
- cd env_name\Scripts
- activate
- pip install Django

Creation of a Project

Navigate to where in the file system you want to store the code (in the virtual environment), and run this command in the command prompt:

```
django-admin startproject project_name
```

OR

```
python -m django startproject project_name
```

Eg.

```
django-admin startproject myProject
```

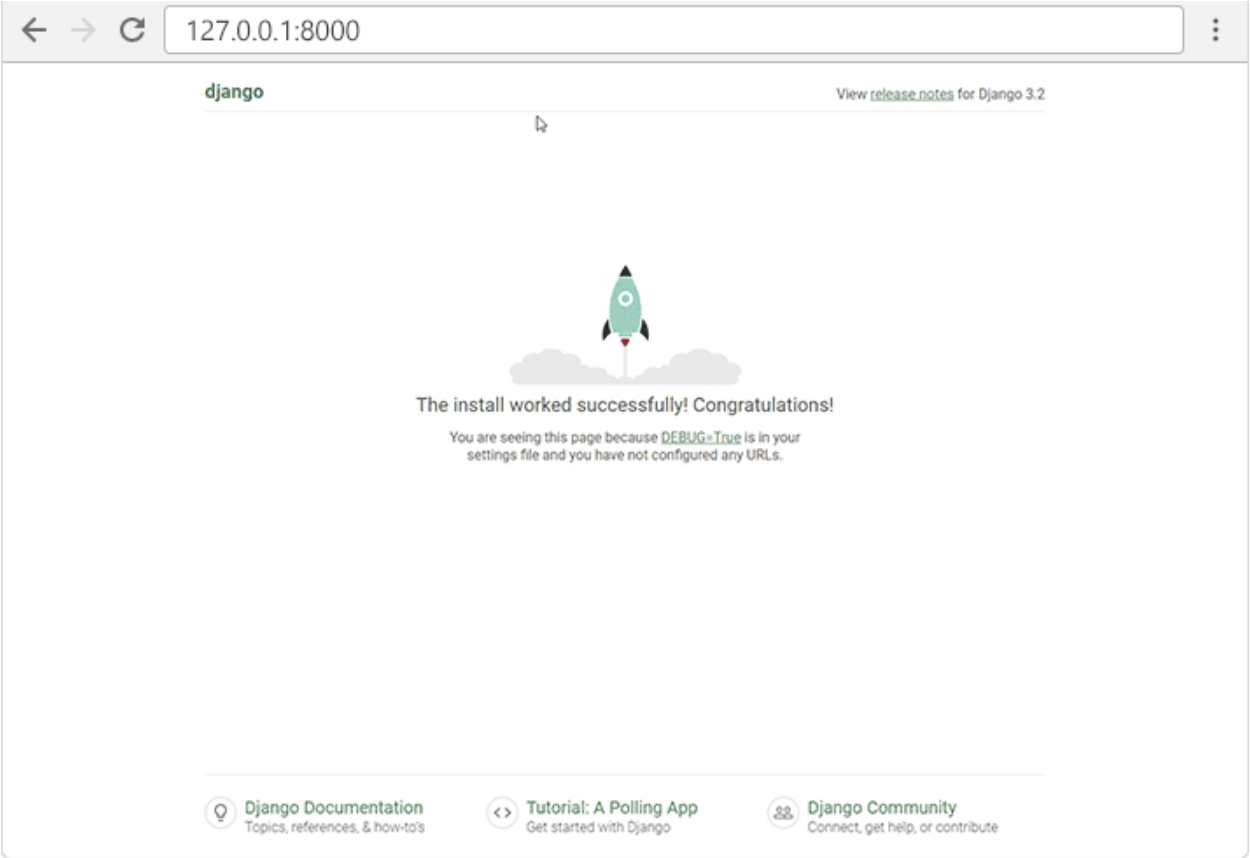
Django creates a myProject folder on my computer, with this content and these files and folders are present with a specific meaning:

```
myProject
  manage.py
  myProject/
    __init__.py
    asgi.py
    settings.py
    urls.py
    wsgi.py
```

Run the Django Project

Navigate to the /myProject folder and execute this command in the command prompt:

```
python manage.py runserver
```



Creation of Apps and their Structure

An app is a web application that has a specific meaning in your project, like a home page, a contact form, or a members database.

Navigate to the myProject (if not done), and run the command below.

python manage.py startapp name_of_app

Eg.

python manage.py startapp myApp

Django creates a folder named myApp in the project, with this content:

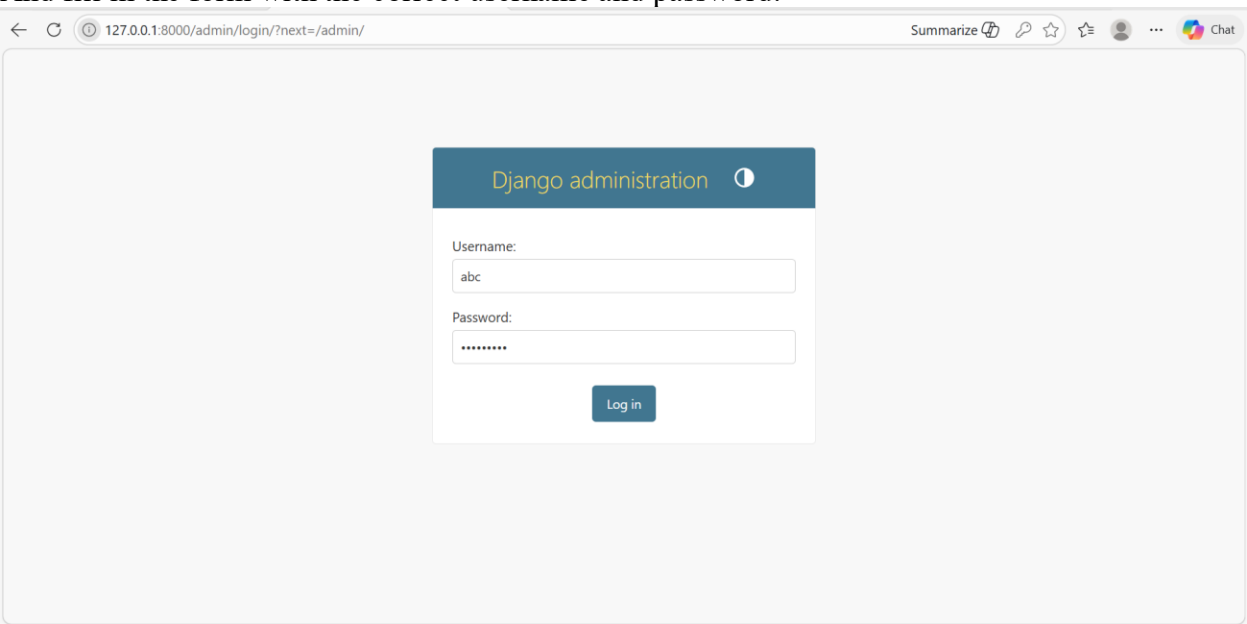
```
myProject
  manage.py
  myProject/
  myApp/
    migrations/
      __init__.py
    __init__.py
    admin.py
    apps.py
    models.py
```

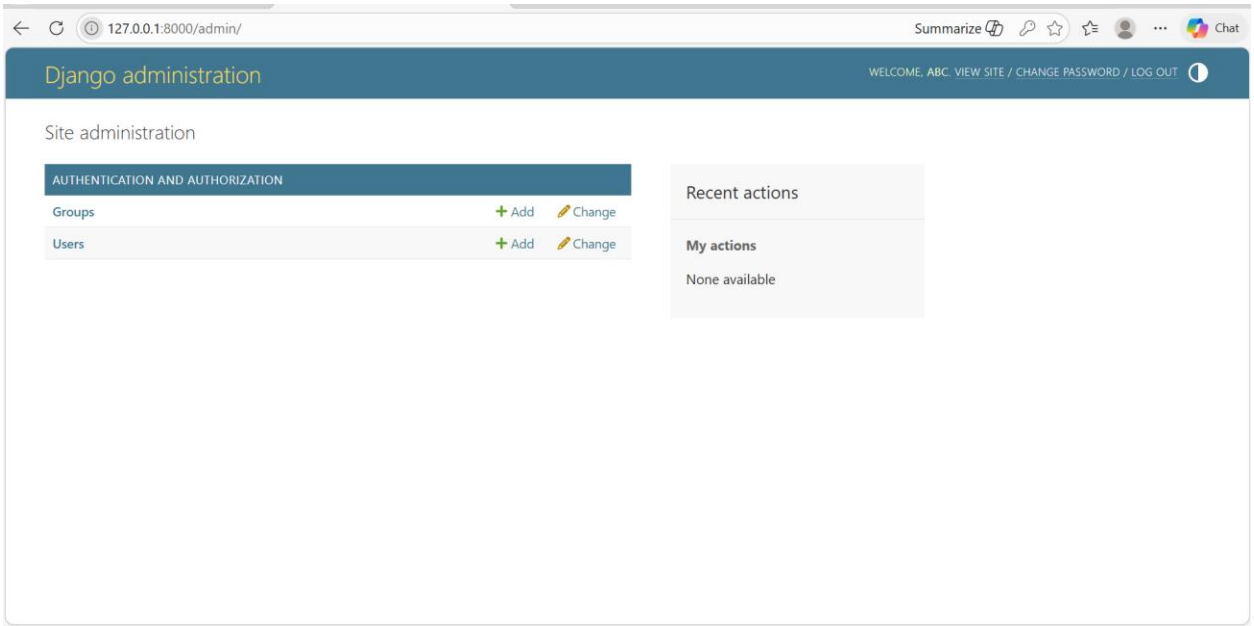
tests.py
views.py

Working with ADMIN Console
To be able to log into the admin application, we need to create a user.
This is done by typing this command in the command view:
python manage.py createsuperuser

Here you must enter: username, e-mail address, (you can just pick a fake e-mail address), and password:
Eg.
Username: abc
Email address: abc@ymail.com
Password:
Password (again):
Superuser created successfully.

Now start the server again:
python manage.py runserver
In the browser window, type 127.0.0.1:8000/admin/ in the address bar.
And fill in the form with the correct username and password:





Creating Views

Django views are Python functions that take http requests and return http response, like HTML documents.

A web page that uses Django is full of views with different tasks and missions.

Views are usually put in a file called **views.py** located on your app's folder.

This is a simple example on how to send a response back to the browser.

Eg.

myProject/myApp/views.py

```
from django.shortcuts import render
from django.http import HttpResponse

def index(request):
    return HttpResponse("Hello world!")
```

URL Mapping

Create a file named **urls.py** in the same folder as the views.py file, and type this code in it:

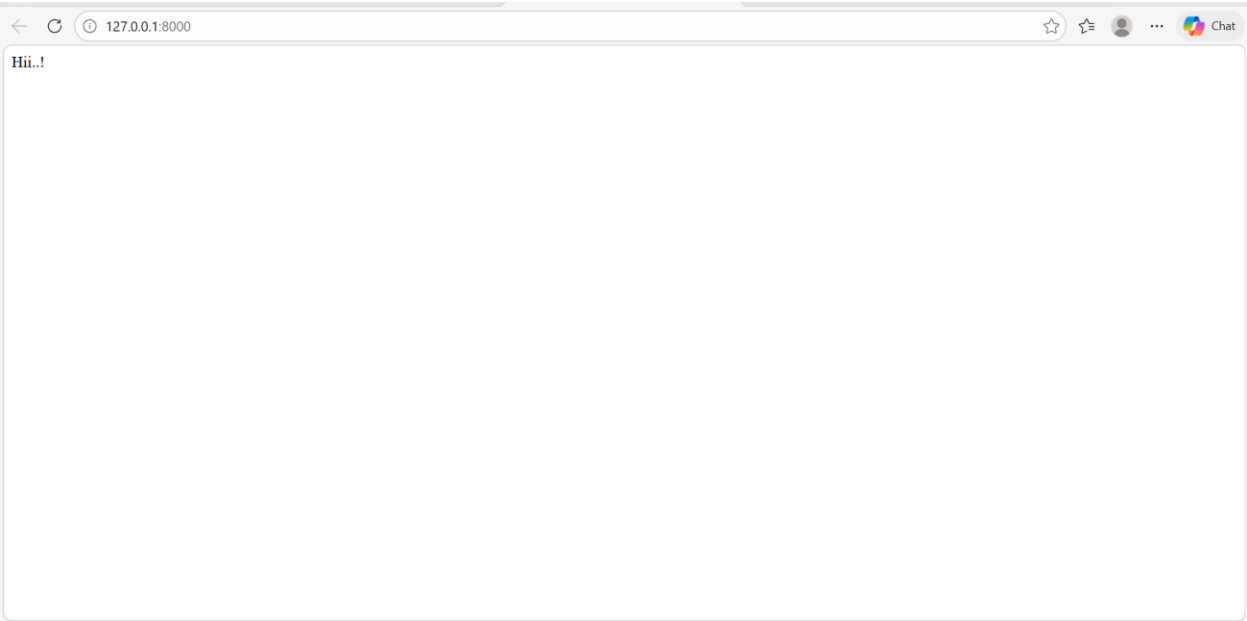
Eg.

myProject/myApp/urls.py

```
from django.urls import path
from . import views

urlpatterns = [
    path('', views.index, name='index'),
]
```

There is a file called **urls.py** on the myProject folder, open that file and add the include module in the import statement, and also add a path() function in the urlpatterns[] list, with arguments that will route users that comes in via 127.0.0.1:8000/.
Eg.
myProject/myProject/urls.py:

```
from django.contrib import admin
from django.urls import include, path

urlpatterns = [
    path('', include('myApp.urls')),
    path('admin/', admin.site.urls),
]
```



Template System
Create a **templates** folder inside the myApp folder, and create a HTML file named **index.html**.
The file structure should be like this:

```
myProject
    manage.py
    myProject/
    myApp/
        templates/
            index.html
```

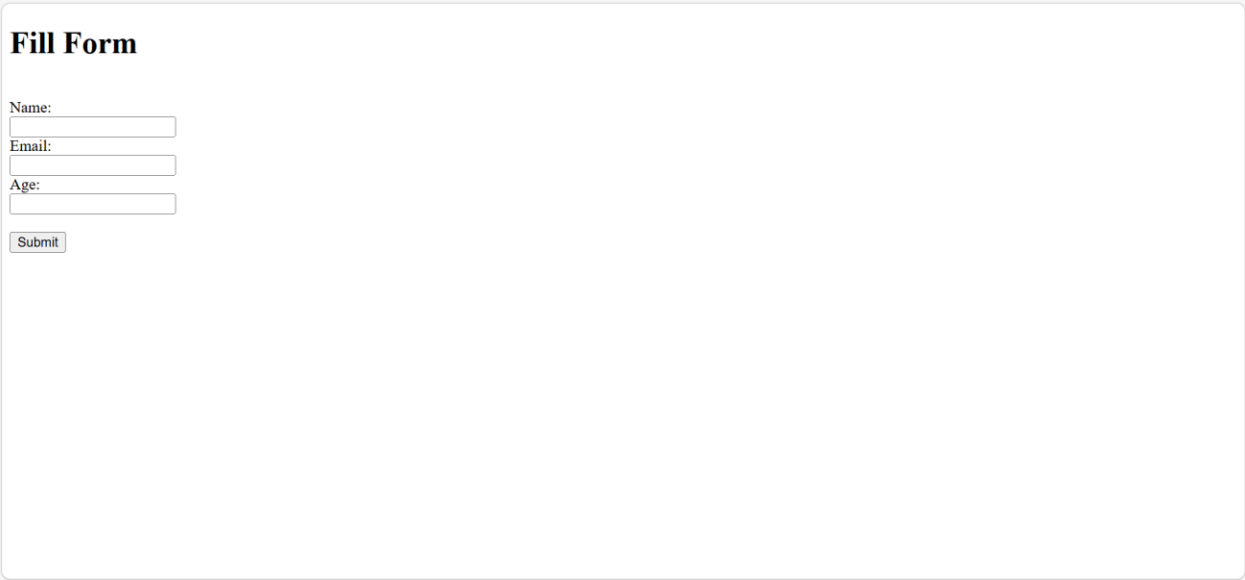
Eg.
myProject/myApp/templates/index.html

```
<!DOCTYPE html>
```

```
<html lang="en">
<head>
   <meta charset="UTF-8">
   <meta name="viewport" content="width=device-width, initial-scale=1.0">
   <title>My Project</title>
</head>
<body>
   <h1>Fill Form</h1>
   <br>
   <form method="POST">
      <label for="name">Name:</label><br>
      <input type="text" name="name"><br>
      <label for="email" name=""email>Email:</label><br>
      <input type="email" name="email"><br>
      <label for="age">Age:</label><br>
      <input type="text" name="age"><br><br>
      <button type="submit">Submit</button>
   </form>
</body>
</html>
```

Open the **views.py** file in the myApp folder, and replace its content with this:
Eg.
myProject/myApp/views.py

```
from django.shortcuts import render

def index(request):
   return render(request, "index.html")
```



Working with Models

- A Django model is a table in your database.
- In Django, data is created in objects, called Models, and is actually tables in a database.
- To create a model, navigate to the **models.py** file in the /myApp/ folder.
- Open it, and add a Member table by creating a Member class, and describe the table fields in it:
  Eg.
  myProject/myApp/models.py

  ```
  from django.db import models

  # Create your models here.
  class Member(models.Model):
    name = models.CharField(max_length=100)
    email = models.EmailField()
    age = models.CharField(max_length=10)
  ```

- Now when we have described a Model in the models.py file, we must run a command to actually create the table in the database.
- Navigate to the /myProject/ folder and run this command:
  python manage.py makemigrations
  python manage.py migrate

Form Processing

- Form processing is the mechanism through which a web application **collects input from users through an HTML form, processes the submitted data on the server, and stores or displays the results**. In Django, this is typically done using **models, views, and templates**.
- In this example, a user fills a form with **name, email, and age**, and the data is stored in the database and then displayed on the same page.
- **Model (Database Structure)**
  The model defines the **structure of the database table**. Django automatically creates the table based on the model fields.
  (Eg. We are using the models.py from previous topic)
- **View (Form Processing Logic)**
  The view handles **receiving form data, inserting it into the database, and retrieving stored records**.
  Eg.
  myProject/myApp/views.py

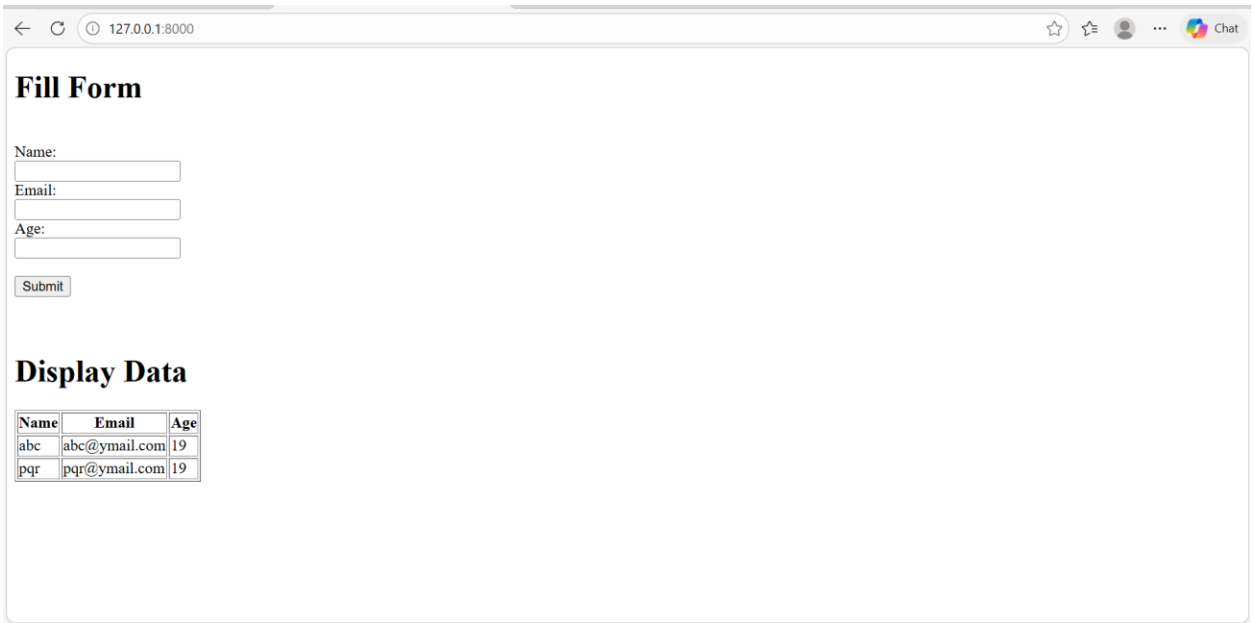
  ```
  from django.shortcuts import render
  from .models import *
  from django.db import connection
  ```

```
# Create your views here.
def index(request):
    if request.method == "POST":
        name = request.POST.get('name')
        email = request.POST.get('email')
        age = request.POST.get('age')

        cursor = connection.cursor()
        cursor.execute('INSERT INTO myApp_member (name, email, age) VALUES (%s,
%s, %s)', [name, email, age])

        data = cursor.execute('SELECT name, email, age FROM
myApp_member;').fetchall()
        return render(request, 'index.html', {'members': data})
    return render(request, 'index.html')
```

- **Template (HTML Form)**
  The template contains the **HTML form for user input** and **table to display stored data**.
  Eg.
  myProject/myApp/templates/index.html

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>My Project</title>
</head>
<body>
    <h1>Fill Form</h1>
    <br>
    <form method="POST">
        {% csrf_token %}
        <label for="name">Name:</label><br>
        <input type="text" name="name"><br>
        <label for="email" name="email">Email:</label><br>
        <input type="email" name="email"><br>
        <label for="age">Age:</label><br>
        <input type="text" name="age"><br><br>
        <button type="submit">Submit</button>
    </form>
    <br><br>
    <h1>Display Data</h1>
    <table border="1">
        <tr>
            <th>Name</th>
            <th>Email</th>
```

```
            <th>Age</th>
        </tr>
        {% for m in members %}
        <tr>
            <td>{{ m.0 }}</td>
            <td>{{ m.1 }}</td>
            <td>{{ m.2 }}</td>
        </tr>
        {% endfor %}
    </table>
</body>
</html>
```



Static media files

- In Django, the **static folder** is used to store files that do not change dynamically and are required for the front-end of the website.
- These files include **CSS stylesheets, JavaScript files, images, fonts, and other media resources** that help design and enhance the user interface of a web application.
- Static files are not processed by Django logic. Instead, they are **served directly to the browser** when requested.
- Common files stored in the static directory include:
    - **CSS files** – for styling web pages
    - **JavaScript files** – for adding interactivity
    - **Images** – such as logos, icons, and backgrounds
    - **Fonts and other media assets**

  Example structure:
  project/

```
static/
    css/
        style.css
    js/
        script.js
    images/
        logo.png
```